Bulk Certificate Generator

Updated Project Setup, API, Testing, and Design Guide

Overview

This FastAPI service creates certificate-generation jobs for multiple recipients, returns HTTP 202, and renders PDFs in a background task. SQLite stores job and recipient state; Jinja2 loads the predefined template; ReportLab writes vector PDFs; files are kept under storage\certificates\\.

Set up the project in PowerShell

```
cd C:\path\to\bulk-certificate-generator
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

If script activation is blocked, run Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass, then activate again. Use Python 3.11 or later.

Run the application

```
python -m uvicorn app.main:app --reload
```

Keep the server window running. Base URL: http://127.0.0.1:8000. Swagger: http://127.0.0.1:8000/docs. Stop the server with Ctrl+C.

Configuration

DATABASE_URL selects the SQLAlchemy database; the default is certificates.db. CERTIFICATE_STORAGE_DIR changes the PDF storage root. MAX_WORKERS controls parallel rendering per job and defaults to 4. Example PowerShell setting: $env:MAX_WORKERS = '4' before starting Uvicorn.

Submit a JSON request

POST /api/v1/jobs accepts event_name, optional issue_date, and recipients. Each recipient needs recipient_name and a valid recipient_email; metadata is optional.

```
$baseUrl = 'http://127.0.0.1:8000'
$body = @{ event_name = 'Python Workshop'; issue_date = '2026-10-10'; recipients = @(@{ recipient_name = 'Alex
Example'; recipient_email = 'alex@example.com'; metadata = @{ grade = 'A' } }) } | ConvertTo-Json -Depth 8
$job = Invoke-RestMethod -Method Post -Uri "$baseUrl/api/v1/jobs" -ContentType 'application/json' -Body $body
$job | Format-List
```

The 202 response includes job_id, status, total_recipients, created_at, and status_check_url. These example names and addresses are fictional.

CSV Upload, Status, and Downloads

Upload a CSV

POST /api/v1/jobs/upload is multipart form data with file, event_name, and optional issue_date. CSV headers can use name or recipient_name and email or recipient_email. Remaining columns become recipient metadata. CSV rows are validated with the same RecipientInput schema as JSON. Invalid CSV, missing required fields, duplicate or blank headers, invalid emails, or malformed rows return HTTP 422 without creating a job.

Example file recipients.csv:

```
name,email,department,grade
Alex Example,alex@example.com,Engineering,A
Jordan Sample,jordan@example.com,Design,B
```

```
curl.exe -X POST http://127.0.0.1:8000/api/v1/jobs/upload `
  -F "event_name=Python Workshop" `
  -F "issue_date=2026-10-10" `
  -F "file=@recipients.csv;type=text/csv"
```

Poll job status

GET /api/v1/jobs/{job_id} returns counts, progress_percentage, timestamps, and per-recipient results. Progress is (completed_count + failed_count) / total_count * 100. Poll until status is terminal: COMPLETED, PARTIALLY_COMPLETED, or FAILED. A successful recipient has a download_url; a failed one has an error_message.

```
$statusUrl = "$baseUrl$($job.status_check_url)"
$status = Invoke-RestMethod -Uri $statusUrl
while ($status.status -in @('PENDING','IN_PROGRESS')) { Start-Sleep 1; $status = Invoke-RestMethod -Uri
$statusUrl }
```

Download a certificate or the job archive

```
$recipient = $status.recipients | Where-Object status -eq 'SUCCESS' | Select-Object -First 1
Invoke-WebRequest -Uri "$baseUrl$($recipient.download_url)" -OutFile
".\certificate-$($recipient.recipient_id).pdf"
Invoke-WebRequest -Uri "$baseUrl/api/v1/jobs/$($job.job_id)/download-all" -OutFile
".\certificates-$($job.job_id).zip"
```

Individual downloads are application/pdf. The ZIP includes every successful certificate and omits failed recipients. A not-yet-ready recipient returns 409; unknown identifiers return 404.

Rate limiting

Each job submission endpoint, JSON POST /api/v1/jobs and multipart POST /api/v1/jobs/upload, has a limit of 10 requests per minute per client IP. Excess requests return HTTP 429. The default SlowAPI storage is in memory and process-local. For multiple API processes or instances, configure shared storage. Behind a reverse proxy, only trust client-IP forwarding from a correctly configured proxy.

Testing, Benchmark, and Design

Run tests

With the virtual environment active, run the full test suite and coverage report:

```
pytest --cov=app
```

Tests use TestClient, an isolated in-memory SQLite database, and temporary storage. Coverage includes JSON/CSV creation, validation, PDF generation, progress, failure isolation, PDF/ZIP retrieval, rate limiting, and the concurrency cap. The verified suite result is 10 passed; latest measured application coverage was 94%.

Worker concurrency

Each job batch uses a ThreadPoolExecutor with at most MAX_WORKERS running or queued recipient render tasks at once. The default is 4. Results and database commits are processed on the worker's main thread, while PDF rendering and file writing run in the bounded pool. The limit is per batch; multiple simultaneous jobs each have their own pool.

Run the 100-recipient benchmark

Start Uvicorn, then in another PowerShell window run:

```
python scripts/benchmark.py
```

The httpx script submits 100 fictional recipients, polls job status to completion, and prints elapsed time and successful certificates per second. Optional controls: --base-url, --poll-interval, and --timeout. Example: python scripts/benchmark.py --base-url http://127.0.0.1:8000 --poll-interval 0.25 --timeout 300.

A verified local run completed 100/100 certificates in 1.74 seconds (57.38 certificates/second). This is a sample measurement, not a performance guarantee.

Key design decisions

202 plus background processing lets clients poll rather than wait for rendering. FastAPI BackgroundTasks is process-local and not durable across a restart; use a persistent task queue when durability or horizontal scaling is required. A per-recipient exception boundary records failures and continues later work. PDFs are stored individually; ZIP creation copies files in chunks into a disk-backed temporary archive to avoid retaining all PDFs in memory.

The API does not currently impose a maximum recipient count. Real capacity depends on request size, storage, worker time, server timeouts, and simultaneous jobs; benchmark measurements should be repeated on the target deployment.